INDEX

INDEX

Playwright (JavaScript)

- Advanced Topics (Part 2)
    1. Network Interception ⭐
    2. API Mocking
    3. Debugging Tools
    4. Trace Viewer
    5. Parallel Execution
    6. Retries & Error Handling

- API Testing with Playwright (Post Advanced Section)
    1. APIRequestContext ⭐
    2. HTTP Methods (GET, POST, PUT, DELETE)
    3. Request Headers
    4. Authentication (Bearer Token)
    5. Response Validation
    6. Status Code Validation
    7. API Chaining ⭐
    8. UI + API Combined Flow ⭐
    9. Best Practices

CORE CONCEPTS

PLAYWRIGHT → CORE CONCEPTS (COMPLETE)

We will cover:

1. Architecture
2. Installation & Project Setup
3. Test Runner (test, expect)
4. Browser Context & Page

🧠1️⃣ PLAYWRIGHT ARCHITECTURE

1️⃣ What is it

Playwright architecture defines how tests interact with browsers using a **Node.js layer + browser engines (Chromium, Firefox, WebKit)**.

2️⃣ Key Components

- Test Script (JS/TS)
- Playwright API
- Browser Server
- Browser Engines
- Browser Context
- Page

3️⃣ How YOU should answer (Interview Style)

> Playwright follows a client-server architecture where test scripts communicate with browser engines through Playwright APIs. It supports multiple browsers like Chromium, Firefox, and WebKit, and uses browser contexts for isolated test execution.

4️⃣ Practical Example

```javascript
test('example', async ({ page }) => {
  await page.goto('https://example.com');
});
```

👉 Script → Playwright → Browser → Execution

5️⃣ Common Mistakes

❌ Saying “Playwright only works on Chrome”
❌ Not mentioning **browser context**
❌ Over-explaining internal working

6️⃣ Expected Questions

- What is Playwright architecture?
- How is it different from Selenium?
- What is browser context?

🧠2️⃣ INSTALLATION & PROJECT SETUP

1️⃣ What is it

Setting up Playwright project using Node.js and installing required dependencies.

2️⃣ Key Components

- Node.js
- npm
- @playwright/test
- Browsers installation
- Project structure

3️⃣ How YOU should answer

> I set up Playwright using Node.js by installing @playwright/test via npm. Then I install browser binaries and create a test folder structure for writing test cases.

4️⃣ Practical Example

```bash
npm init -y
npm install -D @playwright/test
npx playwright install
```

5️⃣ Common Mistakes

❌ Saying “Playwright needs Selenium”
❌ Not knowing commands
❌ Not knowing project structure

6️⃣ Expected Questions

- How do you install Playwright?
- What are prerequisites?
- What is project structure?

🧠3️⃣ TEST RUNNER (test, expect)

1️⃣ What is it

Playwright Test Runner is used to execute tests and perform assertions.

2️⃣ Key Components

- test()
- expect()
- hooks (beforeEach, afterEach)
- fixtures

3️⃣ How YOU should answer

> Playwright provides a built-in test runner where test() defines test cases and expect() is used for assertions. It supports fixtures and hooks for better test management.

4️⃣ Practical Example

```javascript
test('title check', async ({ page }) => {
  await page.goto('https://example.com');
  await expect(page).toHaveTitle(/Example/);
});
```

5️⃣ Common Mistakes

❌ Confusing test() with function
❌ Not knowing expect() usage
❌ Ignoring hooks

6️⃣ Expected Questions

- What is test() in Playwright?
- What is expect()?
- What are hooks?

🧠 4️⃣ BROWSER CONTEXT & PAGE

1️⃣ What is it

Browser context is an isolated session, and page represents a tab.

2️⃣ Key Components

- Browser
- Context
- Page

3️⃣ How YOU should answer

> Browser context is like a separate session similar to incognito mode, and page represents a single tab. This helps in running tests independently without affecting each other.

4️⃣ Practical Example

```javascript
const context = await browser.newContext();
const page = await context.newPage();
```

5️⃣ Common Mistakes

❌ Saying context = browser
❌ Not understanding isolation
❌ Confusing page and context

6️⃣ Expected Questions

- What is browser context?
- Difference between browser and context?
- Why is context important?

LOCATORS

🚀 PLAYWRIGHT → LOCATORS (PART 1)

We'll cover in parts:

1. locator()
2. getByRole() ⭐
3. getByText()
4. CSS & XPath
5. Best Practices

🧠1️⃣ locator()

1️⃣ What is it

`locator()` is a Playwright method used to find elements using selectors like CSS, XPath, id, etc.

2️⃣ Key Components

- CSS selectors (`#id`, `.class`)
- XPath
- Chaining (`locator().locator()`)
- Auto-waiting support

3️⃣ How YOU should answer (Interview Style)

> locator() is a flexible method in Playwright used to identify elements using CSS or XPath selectors. It supports auto-waiting and allows chaining for better element targeting.

4️⃣ Practical Example

```javascript
await page.locator('#username').fill('Kalpesh');
await page.locator('.login-btn').click();
```

5️⃣ Common Mistakes

❌ Using long XPath unnecessarily
❌ Not using chaining
❌ Over-relying on CSS when better options exist

6️⃣ Expected Interview Questions

- What is locator() in Playwright?
- Difference between locator() and getByRole()?
- Why is locator better than findElement (Selenium)?

🧠2️⃣ getByRole() ⭐ (MOST IMPORTANT)

1️⃣ What is it

Used to locate elements based on **ARIA roles (accessibility-based)**.

2️⃣ Key Components

- Role (button, textbox, link)
- Name (visible label/text)
- Accessibility tree

3️⃣ How YOU should answer

> getByRole() is the most recommended locator in Playwright as it uses accessibility roles, making tests more stable, readable, and aligned with real user interactions.

4️⃣ Practical Example

```javascript
await page.getByRole('button', { name: 'Login' }).click();
```

5️⃣ Common Mistakes

❌ Not using role-based locators
❌ Ignoring accessibility
❌ Using XPath instead

6️⃣ Expected Interview Questions

- Why is getByRole preferred?
- What is the ARIA role?
- Difference between getByRole and locator?

🧠 3️⃣ getByText()

1️⃣ What is it

Used to locate elements using **visible text on the page**.

2️⃣ Key Components

- Exact text match
- Partial text match
- Case sensitivity
- Works on visible UI text

3️⃣ How YOU should answer (Interview Style)

> getByText() is used to locate elements based on visible text. It is simple and readable but less stable compared to role-based locators since UI text can change frequently.

4️⃣ Practical Example

JavaScript

```
await page.getByText('Login').click();
await page.getByText('Submit').click();
```

5️⃣ Common Mistakes

❌ Overusing text-based locators
❌ Not handling dynamic text
❌ Using when role-based locator is better

6️⃣ Expected Interview Questions

- When do you use getByText()?
- What are limitations of text locators?

🧠4️⃣ CSS & XPath

1️⃣ What is it

Traditional selector strategies used to locate elements in DOM.

2️⃣ Key Components

- CSS (`#id`, `.class`, `tag`)
- XPath (`//`, `contains()`, `text()`)

3️⃣ How YOU should answer (Interview Style)

> CSS and XPath are fallback locator strategies. CSS is generally faster and preferred, while XPath is useful for complex DOM traversal when other locators are not sufficient.

4️⃣ Practical Example

```javascript
await page.locator('#username').fill('test'); // CSS
await page.locator('//button[text()="Login"]').click(); // XPath
```

5️⃣ Common Mistakes

❌ Writing long and complex XPath
❌ Using XPath when better locators exist
❌ Not optimizing selectors

6️⃣ Expected Interview Questions

- CSS vs XPath?
- Which is faster and why?
- When do you use XPath?

🧠5️⃣ Best Practices ⭐ (INTERVIEW GOLD)

1️⃣ What is it

Guidelines to write **stable, maintainable locators**.

2️⃣ Key Components

- Prefer getByRole()
- Use data-testid
- Avoid XPath
- Keep selectors simple
- Use chaining

3️⃣ How YOU should answer (Interview Style)

> My approach is to prefer role-based locators like getByRole for stability. If not available, I use data-testid or CSS selectors. I avoid XPath unless necessary and focus on maintainable and readable locators.

4️⃣ Practical Example

```JavaScript
await page.getByRole('button', { name: 'Submit' }).click();
await page.locator('[data-testid="login-btn"]').click();
```

5️⃣ Common Mistakes

❌ Using absolute XPath
❌ Writing brittle selectors
❌ Ignoring readability

6️⃣ Expected Interview Questions

- What is your locator strategy?
- Which locator do you prefer and why?
- How do you write stable locators?

Actions & Assertions

PLAYWRIGHT → Actions & Assertions

This section is heavily asked because interviewers want to know:

👉 Can you actually interact with UI properly?
👉 Do you know validations?

We'll cover:

1. Actions
2. Assertions
3. Auto-waiting behavior
4. Best Practices

🧠1️⃣ Actions

1️⃣ What is it

Actions are methods used to interact with web elements like clicking, typing, selecting, hovering, etc.

2️⃣ Key Components

- click()
- fill()
- type()
- check()
- uncheck()
- hover()
- selectOption()
- press()

3️⃣ How YOU should answer (Interview Style)

> Actions in Playwright are used to simulate real user interactions such as clicking buttons, entering text, selecting dropdown values, and hovering over elements. The playwright automatically waits for elements before performing actions.

4️⃣ Practical Example

JavaScript

```javascript
await page.locator('#username').fill('Kalpesh');
await page.getByRole('button', { name: 'Login' }).click();
await page.locator('#country').selectOption('India');
```

5️⃣ Common Mistakes

❌ Using unnecessary waits before actions
❌ Using force click without reason
❌ Confusing fill() and type()

6️⃣ Expected Interview Questions

- Difference between fill() and type()?
- What actions have you used in Playwright?
- How does Playwright handle waits during actions?

🧠2️⃣ Assertions

1️⃣ What is it

Assertions are validations used to verify expected behavior or UI state.

2️⃣ Key Components

- toHaveText()
- toBeVisible()
- toHaveURL()
- toHaveTitle()
- toBeChecked()

3️⃣ How YOU should answer (Interview Style)

> Assertions in Playwright are used to validate application behavior. Playwright provides built-in auto-retrying assertions which improve test stability and reduce flaky tests.

4️⃣ Practical Example

```javascript
await expect(page).toHaveTitle(/Dashboard/);
await expect(page.locator('.success-msg')).toBeVisible();
await expect(page.locator('#name')).toHaveValue('Kalpesh');
```

5️⃣ Common Mistakes

❌ Using manual validations instead of expect()
❌ Not understanding auto-retry behavior
❌ Writing weak assertions

6️⃣ Expected Interview Questions

- What is assertion in Playwright?
- Difference between hard and soft assertion?
- Why are Playwright assertions stable?

🧠3️⃣ Auto-Waiting ⭐ (VERY IMPORTANT)

1️⃣ What is it

The playwright automatically waits for elements to become ready before performing actions or assertions.

2️⃣ Key Components

- Visibility check
- Element enabled state
- Stability check
- Auto-retrying assertions

3️⃣ How YOU should answer (Interview Style)

> One major advantage of Playwright is built-in auto-waiting. Before performing actions, Playwright ensures elements are visible, stable, and actionable, reducing flaky tests and minimizing explicit waits.

4️⃣ Practical Example

```javascript
await page.getByRole('button', { name: 'Submit' }).click();
```

👉 No explicit wait required.

5️⃣ Common Mistakes

❌ Adding unnecessary waitForTimeout()
❌ Using hard waits everywhere
❌ Not trusting Playwright auto-wait

6️⃣ Expected Interview Questions

- What is auto-waiting in Playwright?
- Difference between Playwright and Selenium waits?
- Why does Playwright reduce flaky tests?

🧠4️⃣ Best Practices ⭐

1️⃣ What is it

Guidelines for writing reliable actions and assertions.

2️⃣ Key Components

- Prefer built-in assertions
- Avoid hard waits
- Use stable locators
- Validate meaningful UI behavior

3️⃣ How YOU should answer (Interview Style)

> I avoid hard waits and rely on Playwright's built-in auto-waiting and assertions. I also use stable locators and meaningful validations to improve test reliability.

4️⃣ Practical Example

```javascript
await expect(page.getByRole('heading')).toHaveText('Dashboard');
```

5️⃣ Common Mistakes

❌ Using waitForTimeout() frequently
❌ Weak validations
❌ Over-validating unnecessary UI elements

6️⃣ Expected Interview Questions

- What are best practices for assertions?
- How do you avoid flaky tests?
- Why avoid hard waits?

FRAMEWORK DESIGN

🚀 PLAYWRIGHT → FRAMEWORK DESIGN

This section is one of the **most important areas for SDET interviews** because it reflects:

👉 Real project experience
👉 Scalability thinking
👉 Maintainability practices
👉 Framework architecture understanding

🎯 Topics Covered

1. Page Object Model (POM) ⭐
2. Folder Structure
3. Reusability & Utilities
4. Fixtures
5. Config Management
6. Data-Driven Testing
7. Reporting
8. CI/CD Integration Basics

🧠1️⃣ Page Object Model (POM) ⭐

1️⃣ What is it

POM is a design pattern where web pages are represented as separate classes/files containing locators and reusable methods.

2️⃣ Key Components

- Page classes
- Locators
- Reusable methods
- Separation of test logic and page logic

3️⃣ How YOU should answer (Interview Style)

> Page Object Model is a framework design pattern used to improve maintainability and reusability. In POM, locators and page-specific methods are stored separately from test cases, making framework management easier.

4️⃣ Practical Example

JavaScript

```javascript
class LoginPage {
  constructor(page) {
    this.page = page;
    this.username = page.locator('#username');
    this.password = page.locator('#password');
  }

  async login(user, pass) {
    await this.username.fill(user);
    await this.password.fill(pass);
  }
}
```

5️⃣ Common Mistakes

❌ Writing locators inside test files
❌ Mixing test logic and page logic
❌ Creating very large page classes

6️⃣ Expected Interview Questions

- What is POM?
- Advantages of POM?
- Why use POM in Playwright?
- Difference between POM and traditional framework?

🧠2️⃣ Folder Structure

1️⃣ What is it

Folder structure organizes framework components properly for scalability and maintainability.

2️⃣ Key Components

- pages/
- tests/
- utils/
- fixtures/
- test-data/
- config/

3️⃣ How YOU should answer (Interview Style)

> A proper folder structure helps maintain scalability and readability. I separate pages, test cases, utilities, fixtures, and test data to improve maintainability and collaboration.

4️⃣ Practical Example

```
project-root/
│
├── pages/
├── tests/
├── utils/
├── fixtures/
├── test-data/
├── playwright.config.js
```

5️⃣ Common Mistakes

❌ Keeping everything in one file
❌ No separation of concerns
❌ Improper naming conventions

6️⃣ Expected Interview Questions

- Explain your framework structure
- Why is folder structure important?
- How do you organize test data?

🧠 3️⃣ Reusability & Utilities

1️⃣ What is it

Utilities are reusable helper methods used across the framework to avoid duplicate code.

2️⃣ Key Components

- Common methods
- Helper functions
- Reusable actions
- Generic validations

3️⃣ How YOU should answer (Interview Style)

> I create utility methods for reusable operations like login, waits, screenshots, and common validations to reduce code duplication and improve framework maintainability.

4️⃣ Practical Example

JavaScript

```
async function takeScreenshot(page, name) {
  await page.screenshot({ path: `${name}.png` });
}
```

5️⃣ Common Mistakes

❌ Duplicate code
❌ Large utility files with mixed responsibilities
❌ Hardcoded values

6️⃣ Expected Interview Questions

- What utilities have you created?
- How do you improve framework reusability?
- How do you avoid duplicate code?

🧠4️⃣ Fixtures ⭐

1️⃣ What is it

Fixtures are reusable setup components used to share test setup and dependencies across tests.

2️⃣ Key Components

- Shared setup
- Test isolation
- Dependency injection
- beforeEach equivalent behavior

3️⃣ How YOU should answer (Interview Style)

> Fixtures in Playwright help manage reusable setup logic and dependencies. They improve test isolation and reduce repeated setup code across test cases.

4️⃣ Practical Example

```javascript
test.beforeEach(async ({ page }) => {
  await page.goto('https://example.com');
});
```

5️⃣ Common Mistakes

❌ Repeating setup in every test
❌ Misusing global setup
❌ Not understanding test isolation

6️⃣ Expected Interview Questions

- What are fixtures?
- Why are fixtures useful?
- Difference between hooks and fixtures?

🧠5️⃣ Config Management

1️⃣ What is it

Configuration management controls framework settings like base URL, browser setup, retries, and environments.

2️⃣ Key Components

- playwright.config.js
- Base URL
- Retries
- Parallel execution
- Environment setup

3️⃣ How YOU should answer (Interview Style)

> Playwright configuration helps manage browser settings, execution behavior, retries, and environment-specific values centrally using playwright.config.js.

4️⃣ Practical Example

```JavaScript
use: {
  baseURL: 'https://example.com',
  headless: true
}
```

5️⃣ Common Mistakes

❌ Hardcoding environment values
❌ No retry configuration
❌ Poor environment management

6️⃣ Expected Interview Questions

- What is playwright.config.js?
- How do you manage multiple environments?
- How do you enable retries?

🧠 6️⃣ Data-Driven Testing

1️⃣ What is it

Running the same test with multiple datasets.

2️⃣ Key Components

- JSON data
- Parameterized tests
- External test data

3️⃣ How YOU should answer (Interview Style)

> Data-driven testing helps execute the same test flow with multiple datasets, improving coverage and reducing duplicate test cases.

4️⃣ Practical Example

JavaScript

```javascript
const users = [
  { username: 'admin', password: '1234' },
  { username: 'test', password: '5678' }
];
```

5️⃣ Common Mistakes

❌ Hardcoded test data
❌ Mixing test data and logic
❌ Poor data organization

6️⃣ Expected Interview Questions

- What is data-driven testing?
- How do you manage test data?
- Why use JSON data?

🧠7️⃣ Reporting

1️⃣ What is it

Reporting helps track test execution results and failures.

2️⃣ Key Components

- HTML Reports
- Allure Reports
- Screenshots
- Logs

3️⃣ How YOU should answer (Interview Style)

> Reporting helps analyze execution results and failures efficiently. I use Playwright HTML reports and capture screenshots for failed test cases.

4️⃣ Practical Example

```Bash
npx playwright show-report
```

5️⃣ Common Mistakes

❌ No failure screenshots
❌ Ignoring logs
❌ Poor report readability

6️⃣ Expected Interview Questions

- What reporting tools have you used?
- How do you debug failed tests?
- Why are reports important?

🧠 8️⃣ CI/CD Integration Basics

1️⃣ What is it

Integrating automation execution into CI/CD pipelines for continuous testing.

2️⃣ Key Components

- Jenkins
- GitHub Actions
- Automated execution
- Scheduled runs

3️⃣ How YOU should answer (Interview Style)

> CI/CD integration helps execute automation tests automatically during build or deployment pipelines, improving feedback speed and release quality.

4️⃣ Practical Example

```bash
npx playwright test
```

Executed through Jenkins pipeline.

5️⃣ Common Mistakes

❌ Manual-only execution
❌ No reporting integration
❌ Ignoring pipeline failures

6️⃣ Expected Interview Questions

- How do you integrate Playwright with Jenkins?
- Why is CI/CD important in automation?
- How do you trigger automated execution?

🎯 FINAL INTERVIEW GOLD

Framework Design Priority

1. POM ⭐
2. Reusability
3. Fixtures
4. Config Management
5. Reporting
6. CI/CD

ADVANCED TOPICS (PART 1)

🚀 PLAYWRIGHT → ADVANCED TOPICS (PART 1)

This section covers advanced real-time scenarios asked in SDET and product-company interviews.

🎯 Topics Covered in Part 1

1. Handling Frames
2. Handling Multiple Tabs / Windows
3. File Upload & Download

🧠1️⃣ Handling Frames

1️⃣ What is it

Frames (iframes) are embedded HTML documents inside a web page. Playwright provides specific methods to interact with elements inside frames.

2️⃣ Key Components

- iframe
- frameLocator()
- Nested frames
- Frame isolation

3️⃣ How YOU should answer (Interview Style)

> Frames are embedded web documents inside a page. In Playwright, I use frameLocator() to interact with elements inside iframes because normal locators cannot directly access frame elements.

4️⃣ Practical Example

JavaScript

```javascript
await page.frameLocator('#login-frame')
  .locator('#username')
  .fill('Kalpesh');
```

5️⃣ Common Mistakes

❌ Trying to access iframe elements directly
❌ Using normal locators without frameLocator()
❌ Ignoring nested frames

6️⃣ Expected Interview Questions

- What is iframe?
- How do you handle frames in Playwright?
- Difference between page locator and frame locator?

🧠2️⃣ Handling Multiple Tabs / Windows ⭐

1️⃣ What is it

Handling scenarios where clicking an element opens a new browser tab or window.

2️⃣ Key Components

- context.waitForEvent()
- New page event
- Switching tabs
- Browser context

3️⃣ How YOU should answer (Interview Style)

> In Playwright, multiple tabs are handled using browser context and page events. I use context.waitForEvent('page') to capture and switch to newly opened tabs.

4️⃣ Practical Example

JavaScript

```javascript
const [newPage] = await Promise.all([
  context.waitForEvent('page'),
  page.click('#open-tab')
]);

await newPage.waitForLoadState();
```

5️⃣ Common Mistakes

❌ Not waiting for new page event
❌ Confusing browser context and page
❌ Hardcoding tab switching logic

6️⃣ Expected Interview Questions

- How do you handle multiple tabs in Playwright?
- Difference between context and page?
- How do you switch between tabs?

🧠3️⃣ File Upload & Download

1️⃣ What is it

Handling file upload and file download scenarios in automation testing.

2️⃣ Key Components

- setInputFiles()
- Download event
- File path handling
- Upload validation

3️⃣ How YOU should answer (Interview Style)

> Playwright provides built-in support for file upload and download handling. I use setInputFiles() for uploads and download event listeners for validating downloaded files.

4️⃣ Practical Example

File Upload

```javascript
await page.locator('input[type="file"]')
  .setInputFiles('test-data/sample.pdf');
```

File Download

```javascript
const downloadPromise = page.waitForEvent('download');

await page.click('#download-btn');

const download = await downloadPromise;
```

5️⃣ Common Mistakes

❌ Using OS-level automation unnecessarily
❌ Hardcoding file paths
❌ Not validating downloaded file

6️⃣ Expected Interview Questions

- How do you upload files in Playwright?
- How do you validate downloads?
- Difference between Selenium and Playwright file handling?

🎯 FINAL INTERVIEW GOLD

Advanced Scenario Priority

1. Multiple Tabs ⭐
2. Frames
3. File Upload/Download

Important Concepts to Remember

Concept	Key Point
frameLocator()	Used for iframe handling
context.waitForEvent('page')	Used for new tabs
setInputFiles()	Used for uploads

ADVANCED TOPICS (PART 2)

🚀 PLAYWRIGHT → ADVANCED TOPICS (PART 2)

This section covers high-value advanced Playwright concepts asked in product-company and SDET interviews.

🎯 Topics Covered in Part 2

1. Network Interception ⭐
2. API Mocking
3. Debugging Tools
4. Trace Viewer
5. Parallel Execution
6. Retries & Error Handling

🧠1️⃣ Network Interception ⭐ (VERY IMPORTANT)

1️⃣ What is it

Network interception allows capturing, monitoring, modifying, or blocking API/network requests during test execution.

2️⃣ Key Components

- page.route()
- Request interception
- Response modification
- Blocking requests

3️⃣ How YOU should answer (Interview Style)

> Network interception in Playwright allows us to monitor and modify network requests and responses. It is useful for validating APIs, mocking responses, and handling unstable backend dependencies.

4️⃣ Practical Example

JavaScript

```
await page.route('**/api/users', async route => {
  await route.continue();
});
```

5️⃣ Common Mistakes

❌ Blocking all requests unnecessarily
❌ Incorrect URL patterns
❌ Not understanding request flow

6️⃣ Expected Interview Questions

- What is network interception?
- Why use route() in Playwright?
- Real-time use case of interception?

🧠2️⃣ API Mocking ⭐

1️⃣ What is it

API mocking means simulating backend responses without hitting the actual server.

2️⃣ Key Components

- Mock responses
- route.fulfill()
- Fake API data
- Isolation testing

3️⃣ How YOU should answer (Interview Style)

> API mocking helps test frontend behavior independently from backend systems. In Playwright, I use route.fulfill() to return custom mock responses for stable and isolated testing.

4️⃣ Practical Example

JavaScript

```javascript
await page.route('**/api/login', async route => {
  await route.fulfill({
    status: 200,
    body: JSON.stringify({ success: true })
  });
});
```

5️⃣ Common Mistakes

❌ Mocking unnecessary APIs
❌ Returning invalid response structure
❌ Overusing mocks in all tests

6️⃣ Expected Interview Questions

- What is API mocking?
- Difference between mocking and interception?
- Why use mocked APIs?

🧠 3️⃣ Debugging Tools

1️⃣ What is it

Playwright provides debugging tools to identify failures and analyze test behavior.

2️⃣ Key Components

- page.pause()
- PWDEBUG
- Console logs
- Debug mode

3️⃣ How YOU should answer (Interview Style)

> Playwright debugging tools help analyze failures interactively. I use page.pause(), debug mode, and logs to inspect application state and troubleshoot issues efficiently.

4️⃣ Practical Example

```JavaScript
await page.pause();
```

Run:

```Bash
PWDEBUG=1 npx playwright test
```

5️⃣ Common Mistakes

❌ Leaving pause() in production code
❌ Ignoring logs
❌ Not using debugging tools properly

6️⃣ Expected Interview Questions

- How do you debug Playwright tests?
- What is PWDEBUG?
- How do you analyze failures?

🧠4️⃣ Trace Viewer ⭐

1️⃣ What is it

Trace Viewer is a Playwright tool used to visually analyze test execution step-by-step.

2️⃣ Key Components

- Screenshots
- DOM snapshots
- Network logs
- Execution timeline

3️⃣ How YOU should answer (Interview Style)

> Trace Viewer helps analyze failed test executions visually by providing screenshots, DOM snapshots, and network activity for each step.

4️⃣ Practical Example

Enable tracing:

```JavaScript
use: {
  trace: 'on-first-retry'
}
```

Open report:

```Bash
npx playwright show-trace trace.zip
```

5️⃣ Common Mistakes

❌ Enabling tracing for all executions unnecessarily
❌ Not analyzing traces properly
❌ Ignoring network logs

6️⃣ Expected Interview Questions

- What is Trace Viewer?
- How do you analyze failed tests?
- Difference between report and trace?

🧠 5️⃣ Parallel Execution ⭐

1️⃣ What is it

Running multiple tests simultaneously to reduce execution time.

2️⃣ Key Components

- Workers
- Parallel test execution
- Resource optimization
- Independent tests

3️⃣ How YOU should answer (Interview Style)

> Parallel execution allows multiple tests to run simultaneously using Playwright workers, improving execution speed and CI/CD efficiency.

4️⃣ Practical Example

```JavaScript
workers: 4
```

5️⃣ Common Mistakes

❌ Shared test data issues
❌ Dependent test cases
❌ Environment conflicts

6️⃣ Expected Interview Questions

- What is parallel execution?
- How does Playwright run tests in parallel?
- Challenges in parallel execution?

🧠 6️⃣ Retries & Error Handling

1️⃣ What is it

Retries help rerun failed tests automatically, while error handling manages failures gracefully.

2️⃣ Key Components

- retries
- try-catch
- Failure handling
- Flaky test management

3️⃣ How YOU should answer (Interview Style)

> Retries in Playwright help reduce flaky failures by rerunning failed tests automatically. I also use proper error handling and logging for better debugging.

4️⃣ Practical Example

```JavaScript
retries: 2
```

5️⃣ Common Mistakes

❌ Overusing retries to hide real issues
❌ No proper logging
❌ Ignoring flaky tests

6️⃣ Expected Interview Questions

- What are retries in Playwright?
- How do you handle flaky tests?
- Why should retries be used carefully?

🎯 FINAL INTERVIEW GOLD

Most Important Advanced Topics

1. Network Interception ⭐
2. API Mocking ⭐
3. Parallel Execution ⭐
4. Trace Viewer

Important Commands

Feature	Command
Debug Mode	PWDEBUG=1
Trace Viewer	npx playwright show-trace
Parallel Workers	workers: 4

API TESTING WITH PLAYWRIGH

🚀 PLAYWRIGHT → API TESTING WITH PLAYWRIGHT

This section covers API testing capabilities in Playwright required for modern SDET and product-company interviews.

🎯 Topics Covered

1. APIRequestContext ⭐
2. HTTP Methods (GET, POST, PUT, DELETE)
3. Request Headers
4. Authentication (Bearer Token)
5. Response Validation
6. Status Code Validation
7. API Chaining ⭐
8. UI + API Combined Flow ⭐
9. Best Practices

🧠1️⃣ APIRequestContext ⭐

1️⃣ What is it

APIRequestContext is Playwright's built-in API testing utility used to send HTTP requests directly without browser interaction.

2️⃣ Key Components

- request
- API context
- HTTP requests
- Base URL

3️⃣ How YOU should answer (Interview Style)

> APIRequestContext in Playwright is used for backend/API testing without launching the browser. It helps perform direct HTTP operations like GET, POST, PUT, and DELETE efficiently.

4️⃣ Practical Example

```javascript
const response = await request.get('/users');
```

5️⃣ Common Mistakes

❌ Confusing browser page and API request
❌ Hardcoding endpoints
❌ Not validating responses

6️⃣ Expected Interview Questions

- What is APIRequestContext?
- Why use API testing in Playwright?
- Difference between UI and API testing?

🧠2️⃣ HTTP Methods (GET, POST, PUT, DELETE)

1️⃣ What is it

HTTP methods are operations used to interact with APIs.

2️⃣ Key Components

- GET → Fetch data
- POST → Create data
- PUT → Update data
- DELETE → Remove data

3️⃣ How YOU should answer (Interview Style)

> HTTP methods define the type of API operation. GET retrieves data, POST creates data, PUT updates resources, and DELETE removes resources.

4️⃣ Practical Example

JavaScript

```javascript
await request.get('/users');

await request.post('/users', {
  data: {
    name: 'Kalpesh'
  }
});
```

5️⃣ Common Mistakes

❌ Using wrong HTTP methods
❌ Not validating response codes
❌ Sending incorrect payloads

6️⃣ Expected Interview Questions

- Difference between PUT and POST?
- Which HTTP methods have you used?
- What is idempotency?

🧠 3️⃣ Request Headers

1️⃣ What is it

Headers provide additional information in API requests such as authentication and content type.

2️⃣ Key Components

- Authorization
- Content-Type
- Accept
- Custom headers

3️⃣ How YOU should answer (Interview Style)

> Request headers are used to pass metadata such as authentication tokens and content type information during API communication.

4️⃣ Practical Example

```javascript
await request.get('/users', {
  headers: {
    Authorization: 'Bearer token'
  }
});
```

5️⃣ Common Mistakes

❌ Missing authentication headers
❌ Incorrect content types
❌ Hardcoding sensitive tokens

6️⃣ Expected Interview Questions

- What are request headers?
- Why use Authorization header?
- Difference between headers and body?

🧠 4️⃣ Authentication (Bearer Token) ⭐

1️⃣ What is it

Authentication verifies user identity before accessing protected APIs.

2️⃣ Key Components

- Bearer token
- JWT token
- Authorization header
- Session handling

3️⃣ How YOU should answer (Interview Style)

> Bearer token authentication is commonly used in APIs where a token is passed in the Authorization header to access secured endpoints.

4️⃣ Practical Example

```javascript
headers: {
  Authorization: `Bearer ${token}`
}
```

5️⃣ Common Mistakes

❌ Exposing tokens in code
❌ Expired tokens
❌ Incorrect token format

6️⃣ Expected Interview Questions

- What is bearer token authentication?
- How do you handle tokens securely?
- Difference between session and token auth?

🧠5️⃣ Response Validation

1️⃣ What is it

Validating API response body and returned data.

2️⃣ Key Components

- JSON validation
- Response body
- Field validation
- Schema checks

3️⃣ How YOU should answer (Interview Style)

> Response validation ensures the API returns correct and expected data. I validate fields, response structure, and important business values.

4️⃣ Practical Example

```javascript
const body = await response.json();

expect(body.name).toBe('Kalpesh');
```

5️⃣ Common Mistakes

❌ Validating only status codes
❌ Ignoring response body
❌ Weak assertions

6️⃣ Expected Interview Questions

- How do you validate API responses?
- What validations do you perform?
- Difference between schema and field validation?

🧠 6️⃣ Status Code Validation

1️⃣ What is it

Validating API response status codes.

2️⃣ Key Components

- 200 OK
- 201 Created
- 400 Bad Request
- 401 Unauthorized
- 500 Server Error

3️⃣ How YOU should answer (Interview Style)

> Status code validation confirms whether API operations are successful or failed as expected.

4️⃣ Practical Example

```javascript
expect(response.status()).toBe(200);
```

5️⃣ Common Mistakes

❌ Ignoring failure codes
❌ Hardcoding wrong expectations
❌ Not validating negative scenarios

6️⃣ Expected Interview Questions

- Common HTTP status codes?
- Difference between 401 and 403?
- Why validate status codes?

🧠 7️⃣ API Chaining ⭐

1️⃣ What is it

Using data from one API response in another API request.

2️⃣ Key Components

- Dynamic IDs
- Sequential requests
- Dependency handling

3️⃣ How YOU should answer (Interview Style)

> API chaining is used when one API response provides data required for another API request, such as user IDs or authentication tokens.

4️⃣ Practical Example

```JavaScript
const createUser = await request.post('/users');

const body = await createUser.json();

await request.get(`/users/${body.id}`);
```

5️⃣ Common Mistakes

❌ Hardcoding IDs
❌ Ignoring dependency failures
❌ Poor response handling

6️⃣ Expected Interview Questions

- What is API chaining?
- Real-time example of chaining?
- Why is chaining important?

🧠8️⃣ UI + API Combined Flow ⭐

1️⃣ What is it

Combining API and UI testing together for faster and more efficient automation.

2️⃣ Key Components

- Backend setup
- UI validation
- Faster execution
- Reduced UI dependency

3️⃣ How YOU should answer (Interview Style)

> In real-time projects, I use APIs to create or prepare test data and then validate functionality through UI. This improves execution speed and reduces UI dependency.

4️⃣ Practical Example

JavaScript

```javascript
await request.post('/users', {
  data: {
    name: 'Kalpesh'
  }
});

await page.reload();
```

5️⃣ Common Mistakes

❌ Creating all data through UI
❌ Ignoring backend validation
❌ Slow test setup

6️⃣ Expected Interview Questions

- Why combine API and UI testing?
- Real-time use case?
- Benefits of API-driven setup?

🧠 9️⃣ Best Practices ⭐

1️⃣ What is it

Guidelines for writing stable and maintainable API tests.

2️⃣ Key Components

- Reusable request methods
- Proper validations
- Secure token handling
- Avoid hardcoded data

3️⃣ How YOU should answer (Interview Style)

> I focus on reusable API utilities, proper response validations, secure token management, and meaningful assertions to maintain reliable API automation.

4️⃣ Practical Example

```JavaScript
expect(response.ok()).toBeTruthy();
```

5️⃣ Common Mistakes

❌ Hardcoded test data
❌ No negative testing
❌ Weak assertions

6️⃣ Expected Interview Questions

- API automation best practices?
- How do you secure API tests?
- How do you manage reusable APIs?

🎯 FINAL INTERVIEW GOLD

Most Important API Topics

1. API Chaining ⭐
2. Authentication ⭐
3. Response Validation ⭐
4. UI + API Combined Flow ⭐

Important Validations

Validation	Purpose
Status Code	Operation success
Response Body	Data correctness
Headers	Metadata validation
Schema	Structure validation